QuakeGuard

Distributed Earthquake Early Warning System

Technical Architecture & Protocol Specification

Release: **v2.1.0** (Grafana Observability & System Telemetry)

Core Maintainers: @GiZano, @riccardo0731

Table of Contents

1. Executive Summary & Problem Statement

1.1. The EEW Problem

Earthquake Early Warning (EEW) systems are critical infrastructures designed to detect the initial, less destructive P-waves of an earthquake and issue alerts before the damaging S-waves arrive. Historically, these systems have relied on extremely expensive, professional-grade seismometers deployed sparsely across national territories. This creates blind spots and high latency in areas far from the nearest professional sensor.

QuakeGuard solves this by democratizing seismic detection: it uses a dense network of low-cost IoT edge nodes (ESP32 + MEMS accelerometers) performing local DSP (Digital Signal Processing). By moving the initial detection to the edge and aggregating the triggers in a highly scalable, real-time cloud backend, QuakeGuard achieves the density required for instantaneous local warnings at a fraction of the cost.

1.2. Target Users

- **Civil Protection & Emergency Responders:** For deploying rapid, dense sensor networks around critical infrastructure.
- **Seismological Researchers:** For gathering massive datasets of labeled MEMS accelerograms to complement professional networks.
- **Hobbyists & Citizen Scientists:** For contributing to a community-driven EEW network using inexpensive, off-the-shelf hardware.

1.3. Key Performance Metrics

- **True Positive Rate (TPR):** 80% (validated against INGV professional ground-truth data)
- **False Alarm Rate (FAR):** 0% (achieved via multi-node spatial correlation and strict STA/LTA gating)
- **End-to-End Latency:** < 200ms from edge trigger to mobile alert broadcast

1.4. Differentiators

While commercial and state-sponsored systems like **ShakeAlert**, or crowdsourced applications like **MyShake** and **Earthquake Network (EQN)** rely on smartphone sensors (which must filter out human movement, often restricting data collection to when the device is stationary and charging) or sparse professional seismometers, **QuakeGuard** occupies the “missing middle”: dedicated, rigidly mounted IoT sensors running deterministic C++ DSP on bare-metal RTOS, ensuring zero latency variance and true always-on reliability.

2. System Architecture & Overview

QuakeGuard is a distributed, high-throughput backend system designed for the real-time ingestion, cryptographic validation, and processing of seismic data from IoT devices. It serves as the core infrastructure for an Earthquake Early Warning (EEW) system. The architecture is explicitly designed to handle high-concurrency event firehosing during seismic swarms while maintaining strict security boundaries.

2.1. High-Level Topology

The infrastructure is decoupled into three primary tiers (illustrated in Figure 1):

- **Edge Layer (IoT):** Composed of ESP32-C3 SuperMini microcontrollers interfaced with ADXL345 digital accelerometers. These nodes execute on-device Digital Signal Processing (DSP) using the STA/LTA (Short Term Average / Long Term Average) algorithm.
- **Core Backend & Processing:** A polyglot backend architecture utilizing FastAPI (Python) as the API gateway. Validated data is asynchronously offloaded to a Redis Stream (`readings:stream`) and consumed by horizontally-scalable background workers via consumer groups. The workers persist time-series data into a PostgreSQL/PostGIS database (provisioned as a TimescaleDB hypertable) and trigger area-scoped alerts via Redis Pub/Sub.
- **Client Presentation Layer:** A React Native (Expo) mobile application providing users with real-time seismograph telemetry and instantaneous critical event notifications delivered through WebSockets and native push notifications.
- **Observability Layer:** A Grafana instance automatically provisioned via `docker-compose.yml` with a JSON dashboard and TimescaleDB datasource, providing system telemetry (latency, RSSI, free heap).
- **Device Onboarding:** A WiFiManager-based Captive Portal (`QuakeGuard-Setup` SSID) serves the mobile APK download link and displays the ECDSA public key for zero-touch enrollment. Additionally, the system features a Dynamic Demo Onboarding mode where a QR code is generated in the terminal to configure the mobile app URL without recompilation. A hardware reset via the BOOT button (GPIO 0, 5-second hold) re-enters AP mode for credential recovery.
- **Over-the-Air Updates:** A custom React Native hook (`useUpdateChecker`) polls GitHub Releases for new APK versions and triggers a native sideload prompt, eliminating manual update distribution.

2.2. Data Plane and Control Plane

Following the v1.1.0 cloud migration, the architecture strictly separates the data and control pipelines (see Figure 2 for the core container topology):

- **Data Plane (Telemetry):** Flows exclusively through a local Eclipse Mosquitto broker on port 1883. A Python-based MQTT bridge (`mqtt_subscriber.py`) subscribes to the `quakeguard/telemetry` topic and forwards payloads to the internal FastAPI ingestion pipeline via HTTP POST.
- **Control Plane (Provisioning & Management):** Device onboarding, cryptographic handshakes, and REST retrieval operations are routed through an HTTPS tunnel to the FastAPI endpoints (e.g., `/devices/register`). In development the tunnel is a **Cloudflare quick tunnel** (`cloudflared tunnel --url http://localhost:8000`); production should use a real HTTPS domain. The ngrok free-tier edge is not used because its bot-protection terminates ESP-IDF (mbedTLS) TLS handshakes via JA3 fingerprinting **before** any HTTP header can be read, so IoT clients never reach the backend.

2.3. Key Design Principles

- **Zero-Trust Security:** Every telemetry payload must be cryptographically signed using an ECDSA (NIST256p) private key stored securely in the ESP32′s Non-Volatile Storage (NVS). The backend validates these signatures (SHA-256) and device timestamps to prevent spoofing and replay attacks.

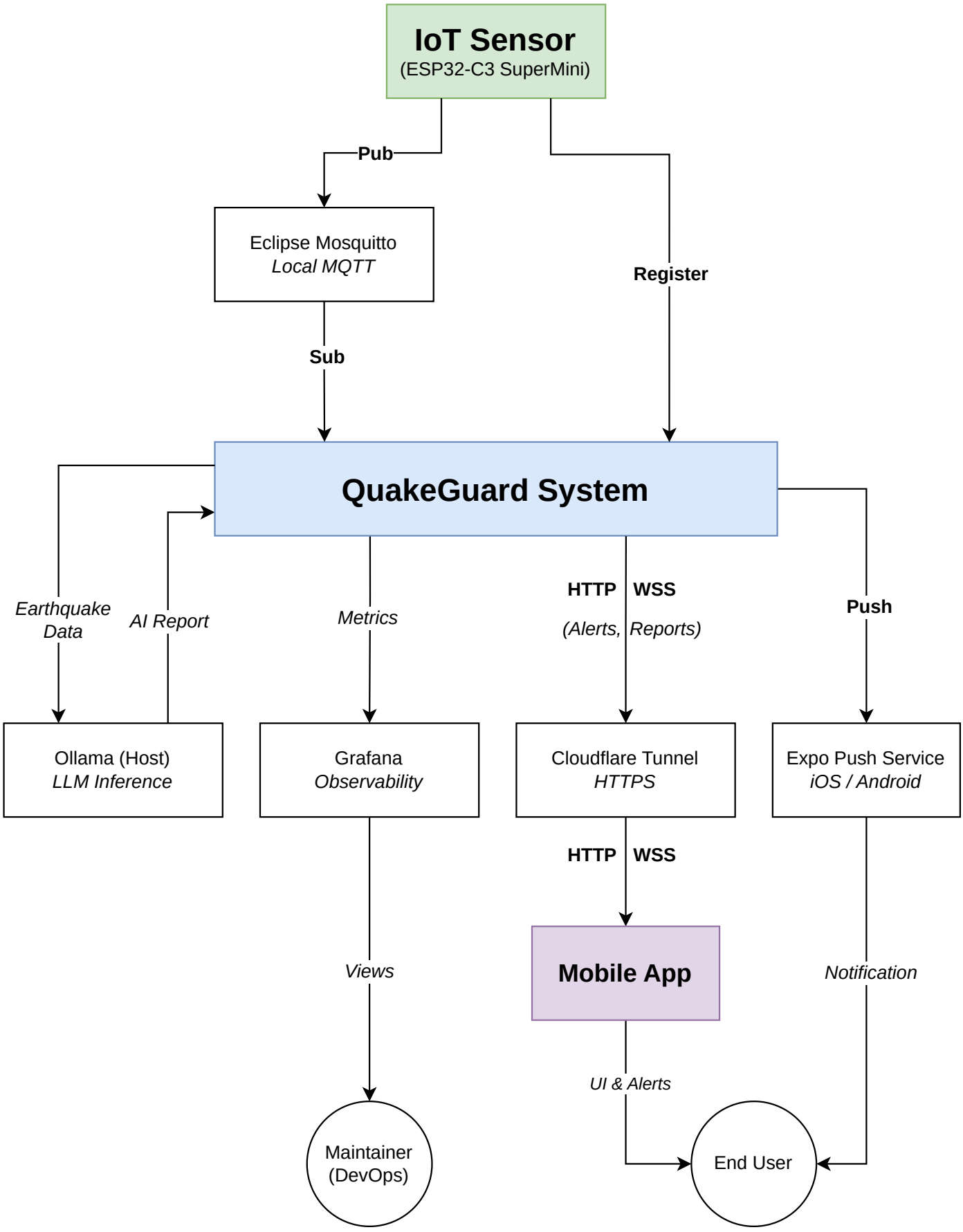

Figure 1: *High-Level Architecture Context Diagram*

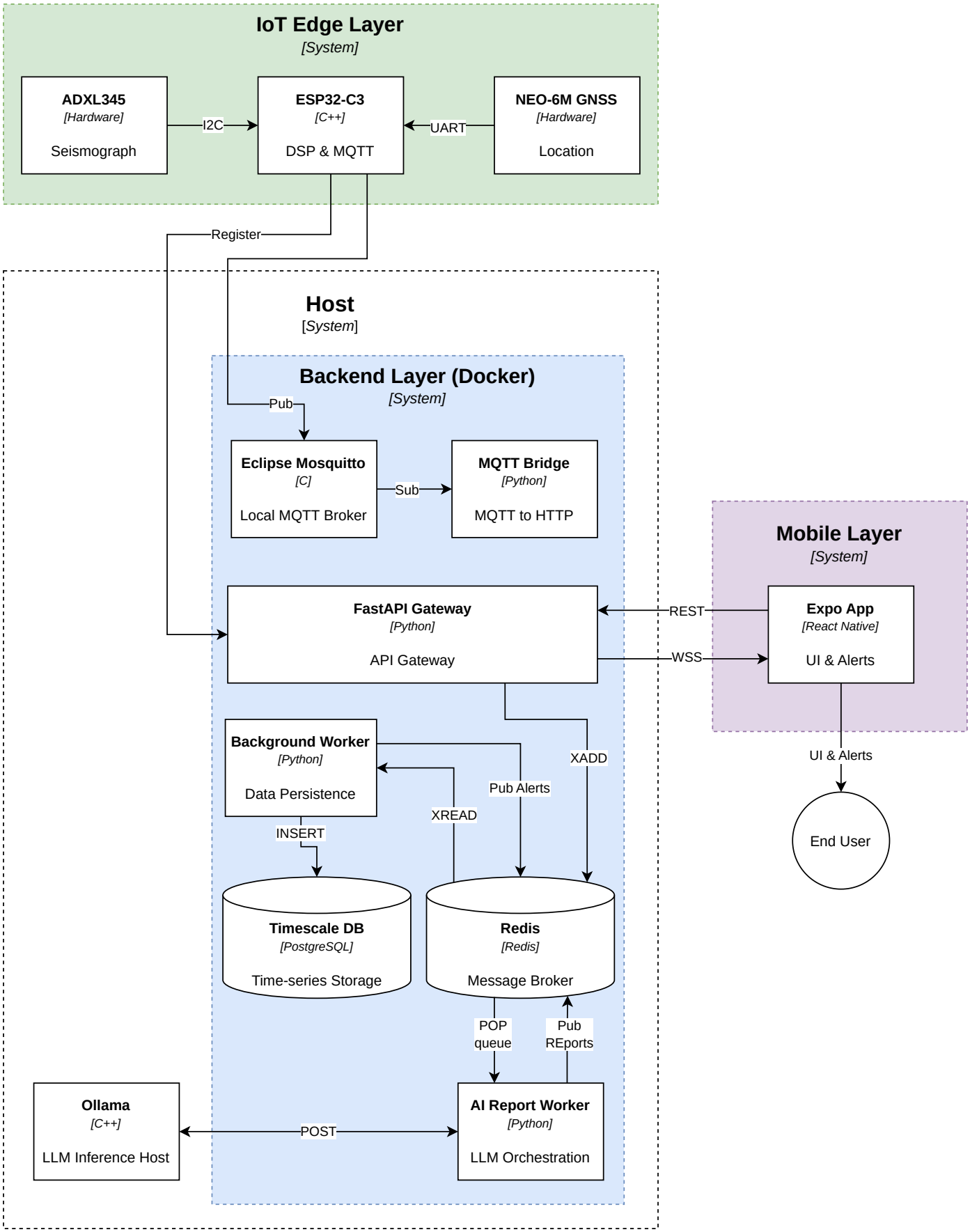

Figure 2: *High-Level Architecture Container Diagram*

- **Asynchronous Decoupling:** The ingestion API is strictly non-blocking. Validated payloads are immediately pushed to the Redis Stream, allowing the gateway to acknowledge the IoT device in milliseconds while background workers handle heavy database transactions and magnitude estimations.
- **Spatial Awareness:** Leveraging PostGIS, the system automatically assigns newly provisioned sensors to geographic zones using polygon containment. A geohash-based Redis index (`zoneindex:<geohash>`) pre-computed at seed time provides a fast-path for coordinate-to-zone resolution, with PostGIS `ST_Contains` as the authoritative fallback. This enables targeted, geographically bounded alert broadcasting with per-area cooldowns.

3. Hardware & Edge Computing (ESP32-C3)

The edge layer of QuakeGuard is designed to operate on resource-constrained microcontrollers, specifically targeting the ESP32-C3 SuperMini platform (RISC-V architecture). The node interfaces with an ADXL345 digital accelerometer to acquire, filter, and analyze seismic data locally, transmitting only cryptographically signed anomaly reports to conserve bandwidth.

3.1. Hardware Configuration & Interfacing

Due to the specific physical layout of the ESP32-C3 SuperMini, the I2C bus is software-mapped to non-standard GPIO pins:

- **SDA (Data):** Mapped to GPIO 7, requiring internal pull-up resistors.
- **SCL (Clock):** Mapped to GPIO 8, requiring internal pull-up resistors.
- **Power Constraints:** The ADXL345 is powered strictly via the 3.3V rail; applying 5V will result in immediate hardware damage.

The sensor operates at a 100Hz sampling rate (`ADXL345_DATARATE_100_HZ`) with a measurement range of $\pm 16G$ (`ADXL345_RANGE_16_G`). To prevent I2C bus race conditions during startup, the sensor object is instantiated dynamically in memory only after the hardware bus is fully stabilized.

3.2. Digital Signal Processing (DSP) & Calibration Pipeline

Raw acceleration data undergoes multiple stages of local processing and calibration before being evaluated:

- **Static Offset Calibration:** Upon boot, the sensor performs a static 100-sample averaging sequence while the node is at rest. The calculated positional offsets are written directly to the ADXL345 hardware registers (`OFSX`, `OFSY`, `OFSZ`), inherently zeroing the sensor at the hardware level and removing physical mounting biases.
- **High-Pass Filter (HPF):** A digital filter (`HPF_ALPHA = 0.9f`) isolates the dynamic vibration data by subtracting the static DC component (Earth's gravity, approx. 9.81 $\frac{m}{s^2}$).
- **Noise Gate:** Micro-vibrations below the empirical threshold of 0.04G are clamped to zero to prevent false positives generated by electrical noise.
- **Dropout Protection:** The firmware automatically drops frames reporting near-zero absolute acceleration (less than 2.0 $\frac{m}{s^2}$ prior to filtering), effectively mitigating corrupted readings caused by temporary I2C wire disconnects.

3.3. STA/LTA Seismic Detection Algorithm

QuakeGuard implements the industry-standard Short-Term Average / Long-Term Average (STA/LTA) algorithm. The implementation utilizes a custom, memory-efficient `RingBuffer` template class in C++ to maintain rolling sums, allowing $O(1)$ average calculations.

- **Short-Term Window (STA):** 100 samples (1 second of telemetry), representing the immediate seismic transient.

- **Long-Term Window (LTA):** 1000 samples (10 seconds of telemetry), representing the background noise floor.
- **Trigger Condition:** An earthquake is registered when the STA/LTA ratio exceeds `1.8f`, provided the STA absolute value is also above the noise floor.

3.4. FreeRTOS Task Architecture

The firmware utilizes FreeRTOS to decouple time-critical sensor acquisition from network latency.

- **`sensorTask`:** Pinned to a high priority, it wakes exactly every 10ms to poll the ADXL345 and execute the DSP pipeline. Upon detecting a seismic event, it pushes a `SeismicEvent` struct to the `eventQueue`.
- **`networkTask`:** Operates asynchronously at a lower priority. It consumes the `eventQueue`, synchronizes the event timestamp via NTP (`pool.ntp.org`), signs the payload, and dispatches the MQTT message. This design ensures that network blocking or Wi-Fi reconnects never halt the continuous monitoring of the accelerometer.
- **`gnssTask` (optional):** When the GNSS subsystem is compiled in, a dedicated task (priority 2) continuously feeds NMEA frames into the parser so the coordinate pipeline is always live.

3.5. Zero-Trust Serial Fallback

The edge node tolerates complete MQTT/WiFi loss without losing cryptographically-attested telemetry. The `networkTask` becomes a **first-available-path** dispatcher: `MQTT → USB CDC Serial → Retention Ring`.

- **Framing:** `SerialFallback.h` builds `[QG:FB]{...}` frames whose JSON is **byte-identical** to the MQTT payload (`value`, `sensor_id`, `device_timestamp`, `signature_hex`). The marker lets the host bridge filter CDC noise without parsing every line.
- **Host-aware routing:** `Serial.isConnected()` (HWCDC, `ARDUINO_USB_CDC_ON_BOOT=1`) distinguishes a real USB host from a power-only charger; with no host the event is retained rather than written to a dead port. The pure routing function `decidePath(mqttReachable, usbHostPresent, timeValid)` is shared between the firmware and the host SIL test suite.
- **Retention & drain:** A bounded `RetentionRing<100>` holds events in FIFO order (oldest overwritten on overflow). When a path becomes reachable the ring is drained in order; each retained event is **re-signed** with the current wall time at drain so the backend ±300 s anti-replay window accepts the retransmission.
- **Offline wall clock:** NTP is opportunistic (`pool.ntp.org` via `configTime`). At the first successful sync the firmware anchors `epochAtSync + millis()`; subsequent timestamps remain valid after WiFi drops and no frame is emitted before `timeValid`.
- **Host bridge:** `firmware/tools/serial_bridge.py` reads `/dev/ttyACM0`, filters `[QG:FB]` lines via `parse_frame()`, validates the ingestion URL against SSRF (`_validate_api_url`), and POSTs to `/readings/` with `X-API-Key` — the same forwarding path as `mqtt_subscriber.py`. A `test_serial_fallback.cpp` suite covers `buildSerialFrame`, `decidePath` and ring semantics on the host.

3.6. Diagnostic LED Indicators

The ESP32-C3 firmware drives two diagnostic LEDs to provide immediate visual feedback of the node's operational state without requiring a serial monitor:

- **Boot Test:** Upon power-up, both the Blue and Red LEDs blink simultaneously twice to verify wiring integrity before the FreeRTOS tasks start.
- **Solid Blue:** The node is fully online, with both Wi-Fi connected and an active MQTT session established with the broker.

- **Single-Blinking Blue:** Wi-Fi is connected, but the MQTT broker is unreachable. The node is attempting opportunistic reconnection.
- **Double-Blinking Blue:** Wi-Fi connection is lost. The node is actively scanning and attempting to re-associate with the AP.
- **Solid Red (3-second pulse):** A seismic event (earthquake) was successfully detected by the STA/LTA algorithm. This overrides any blue LED state.
- **Serial Fallback Indication:** If a seismic event occurs while the Blue LED is blinking (no MQTT), the Red LED will still pulse. This specific combination (Blinking Blue + Solid Red pulse) visually confirms to the operator that the node has fallen back to writing the cryptographically-signed event to the USB CDC Serial port or parking it in the Retention Ring.

3.7. PCB Design & Physical Assembly (v2.0.0)

With the v2.0.0 milestone, the QuakeGuard hardware has matured from a breadboard prototype to a custom Printed Circuit Board (PCB), designed in KiCad. The `hardware/` directory contains the complete schematic, PCB layout, and manufacturing Gerber files.

- **Layout Integration:** The PCB provides dedicated footprints for the ESP32-C3 SuperMini (designed with footprint headroom to support a future ESP32-S3 variant for edge AI), the ADXL345 accelerometer (powered correctly at 3.3V), and the optional u-blox GNSS receiver.
- **Signal Integrity:** The I2C lines (SDA on GPIO 7, SCL on GPIO 8) include proper hardware pull-up resistors on the PCB to guarantee stable communication at 100Hz without relying on the weak internal MCU pull-ups.
- **External Interfacing:** A J4 header exposes the GNSS UART lines (RX GPIO 5, TX GPIO 4, PPS GPIO 2) and power rails, allowing modular connection of the GPS antenna for accurate time synchronization and epicentral triangulation.
- **Bill of Materials (BOM):** The exact hardware components required to assemble the QuakeGuard PCB (including the ESP32-C3, ADXL345, optional GNSS, and THT passives) are exported from KiCad and documented in the official project Wiki under the `Bill of Materials` page (see Figure 2.1).

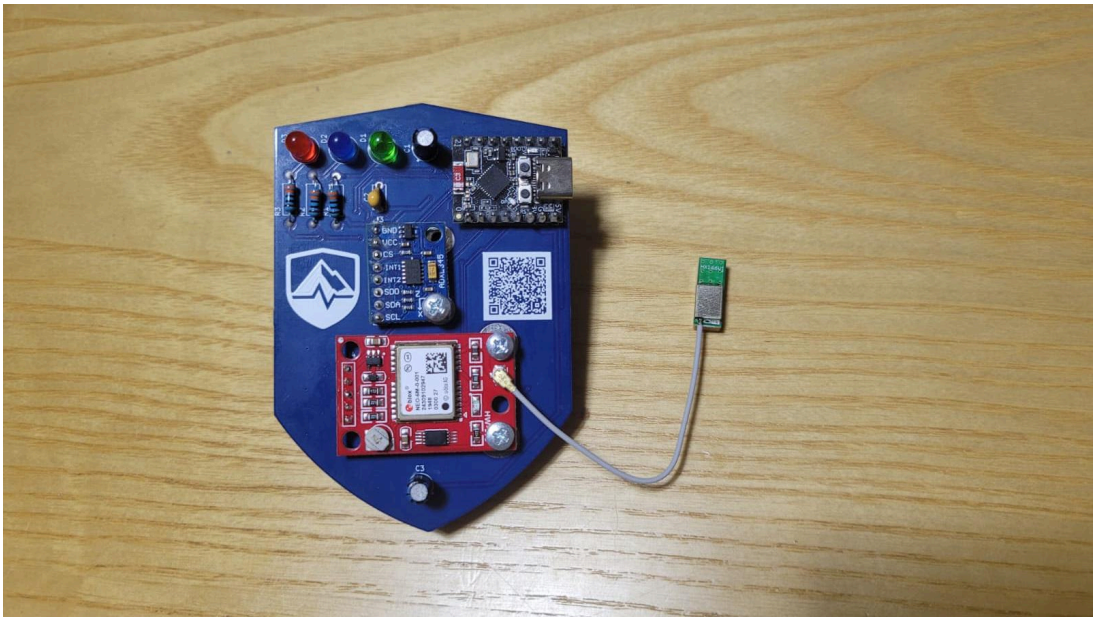

Figure 2.1: *The QuakeGuard v2.0.0 fully assembled PCB, featuring the ESP32-C3 SuperMini, the ADXL345 accelerometer, and the u-blox GNSS module soldered into their dedicated footprints.*

3.8. GNSS Subsystem & NTP Discipline (v2.0.0)

The firmware features an optional GNSS module: an optional module parses NMEA data from a u-blox / NEO-6M / NEO-M8N receiver over the secondary UART (RX GPIO 5, TX GPIO 4, 9600 baud on the JLCPCB — `J4-3→GPIO5`, `J4-4→GPIO4`, `J4-5→GPIO2 PPS` for v2.0.0) at 9600 baud. The module is compiled **only** when `GNSS_ENABLED=1` is present in `esp32_config.env`, so the default build stays hermetic and pulls no `TinyGPSPlus` dependency. Defaults in `GnssModule.h` are `RX 5 / TX 4` to match the fabricated PCB; they remain overridable via `GPS_SERIAL_RX_PIN`/`TX_PIN` in `esp32_config.env` through `extra_script.py`.

- **Last-Known Fix Persistence:** Every reliable fix is saved into NVS (namespace `quake-gnss`, at most once per 60 seconds to limit flash wear). Provisioning therefore reports real coordinates even before the first fix after a cold boot.
- **Staleness Handling:** A live fix older than 10 seconds is treated as stale and the firmware falls back to the last-known value.
- **Provisioning Integration:** During `/devices/register`, the node reports the live GNSS fix when available, else the last-known-from-NVS fix; when neither exists it uses `GNSS_FALLBACK_LAT`/`LON` from `esp32_config.env` if defined (indoor testing), otherwise coordinates are omitted so the backend can assign the zone once placed.
- **NTP + PPS Discipline:** The core feature of v2.0.0. The pulse-per-second (PPS) hardware interrupt (GPIO 2) records the precise `millis()` at the start of each UTC second. The `networkTask` synchronizes via NTP and then disciplines the internal `epochAtSync` directly to the last PPS interrupt, ensuring stratum-1-like millisecond precision for all seismic telemetry.

3.9. Software-in-the-Loop (SIL) Validation

To ensure the theoretical STA/LTA model translates correctly to the real world, the QuakeGuard firmware edge core (`DetectionCore.h`) is tested headlessly against a Python-based Software-in-the-Loop pipeline.

- **Open Data Integration:** The pipeline dynamically queries the public INGV FDSN web service via ObsPy, fetching raw waveforms from the `IV` and `MN` open networks for specific historical baselines (e.g., L'Aquila 2009, Amatrice 2016, Emilia 2012).
- **DSP Simulation:** Raw data is deconvolved to physical acceleration ($\frac{m}{s^2}$), causal bandpass-filtered (1-20 Hz), and resampled to exactly 100 Hz to simulate the ADXL345 physical output.
- **Algorithmic Certification:** The multidimensional calibration sweep proves the current STA/LTA threshold (`ratio=2.4`, `floor=0.02G`) yields a theoretical 80% True Positive Rate with a 0.0% False Alarm Rate against the validation dataset, demonstrating maximal rejection of distant micro-seismicity (like Salizzole at 100km) while triggering on near-fault impulses within 3 seconds of the P-wave arrival. (Note: The firmware deployed on edge nodes uses a more sensitive operational threshold of `1.8f` to ensure no P-waves are missed in the field, whereas the stricter ratio of `2.4` is used specifically in the offline SIL calibration to certify a theoretical 0% False Alarm Rate bound, as shown in Figure 2.2.)

4. Cryptographic Security & Provisioning

QuakeGuard implements a Zero-Trust security model for its IoT edge nodes. To prevent rogue devices from injecting false seismic data (spoofing) or re-transmitting old events (replay attacks), the system relies on asymmetric cryptography and strict temporal validation.

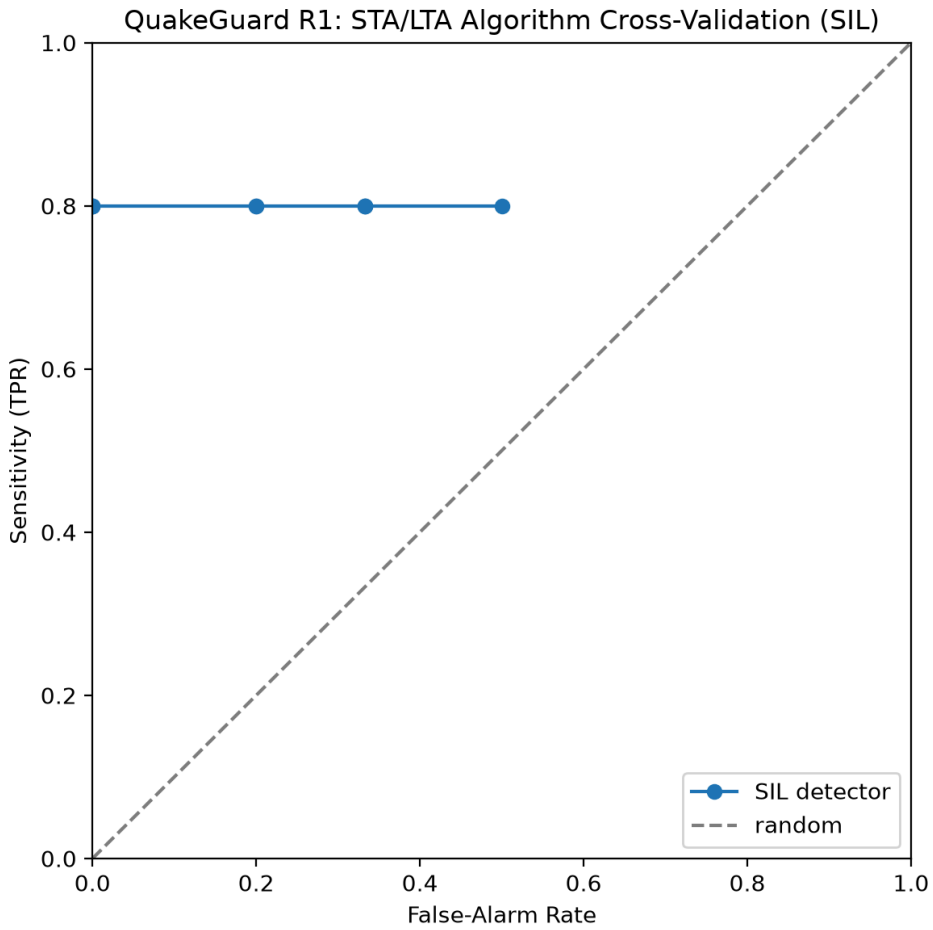

Figure 2.2: *Receiver Operating Characteristic (ROC) curve of the edge STA/LTA algorithm. This represents a theoretical, preliminary MVP validation based on 5 historical events. The curve reaches a hard upper bound at 0.8 (80%) True Positive Rate (Sensitivity). This limit represents a highly desirable geophysical outcome: 4 out of 5 historical events are perfectly identified without any false alarms (FAR = 0.0), while the 5th event (a weak micro-seismicity recorded at >100km distance) correctly fails to trigger the algorithm, proving the firmware's robustness against distant noise.*

4.1. Cryptographic Identity (ECDSA)

Upon its first boot, the ESP32-C3 utilizes the `mbedtls` library to generate a unique Elliptic Curve Digital Signature Algorithm (ECDSA) key pair using the NIST P-256 curve (`secp256r1`).

- **Private Key:** Stored securely and permanently in the device's Non-Volatile Storage (NVS). It never leaves the device and is used exclusively to sign outgoing telemetry.
- **Public Key:** Extracted in DER format, converted to a hexadecimal string, and acts as the unforgeable cryptographic identity of the sensor within the backend database.

4.2. Captive Portal Onboarding

To seamlessly connect the hardware identity with the user's mobile app, the ESP32 hosts an embedded HTTP Captive Portal during its first boot. Users connecting to the `QuakeGuard-Setup` network are immediately presented with a custom HTML page containing two crucial elements:

- A direct download link for the QuakeGuard Mobile App (APK), dynamically resolving to the latest GitHub release.

- The sensor's hexadecimal ECDSA Public Key, generated by `mbedtls` and displayed in clear text.

This allows the user to easily copy the hardware's cryptographic identity and paste it directly into the mobile app's enrollment flow, eliminating complex manual extractions.

4.3. Automated Provisioning Handshake

Before transmitting any seismic data, an unregistered sensor must complete an automated handshake with the Control Plane (detailed in Figure 3.1 and Figure 3.2):

1. The device sends a POST request to the `/devices/register` endpoint, providing its generated `public_key_hex`, MAC address, GPS coordinates, and a hardcoded `ENROLLMENT_TOKEN`.
2. The backend validates the factory enrollment token to ensure the device is authorized to join the network.
3. Using a geohash-based Redis fast-path index with an authoritative PostGIS fallback (`ST_Contains`), the backend spatially evaluates the provided GPS coordinates against the predefined zones and assigns the sensor to the smallest containing geographic polygon. When a GNSS module is attached, the coordinates are the live fix (or the last-known fix persisted in NVS); otherwise the node reports a hardcoded placeholder until provisioned in place.
4. A unique `sensor_id` is returned to the device, which saves it to NVS for all future communications.

4.4. Payload Authentication & Integrity

When a seismic event triggers the DSP pipeline, the firmware constructs a string containing the measured magnitude and the current timestamp (format: `value:timestamp`). Under normal operation the timestamp is the NTP-synchronized wall time; when the serial fallback drains the retention ring it is the software wall clock `epochAtSync + millis()` at drain time, and the payload is **re-signed** with the current time so the backend's replay window accepts it. This string is hashed via SHA-256 and signed with the device's private key.

The resulting JSON payload includes the data, the timestamp, and the `signature_hex` — identical on the MQTT and serial data planes (`[QG:FB]` frames carry the same JSON behind the marker). Once received by the backend, the `validate_iot_payload` dependency pipeline enforces four security gates:

- **API Key Verification:** Validates the `X-API-Key` header using a constant-time comparison (`hmac.compare_digest`) to prevent timing attacks.
- **Sensor Status:** Confirms the sensor ID exists and is marked as `active` in the PostgreSQL database.
- **Anti-Replay Protection:** Compares the device's timestamp against the server's UTC time. Payloads older than a 300-second threshold are outright rejected with a `403 Forbidden` error.
- **Signature Verification:** The backend utilizes the Python `cryptography` library to verify the ECDSA signature against the public key associated with that sensor. The implementation robustly supports both DER-encoded signatures (native to MbedTLS) and raw concatenated $r \parallel s$ signatures.

5. Data Plane & Message Broker (MQTT)

With the release of v2.1.0, QuakeGuard migrated its Data Plane from a cloud-based MQTT architecture to a resilient, local-first infrastructure. This decoupling ensures that high-frequency seismic telemetry is handled by a dedicated message broker optimized for IoT, freeing the edge nodes from the overhead of HTTP round-trips while guaranteeing resilience against WAN outages.

5.1. Eclipse Mosquitto Infrastructure

The core of the Data Plane is a local Eclipse Mosquitto broker, orchestrated natively within the Docker Compose stack.

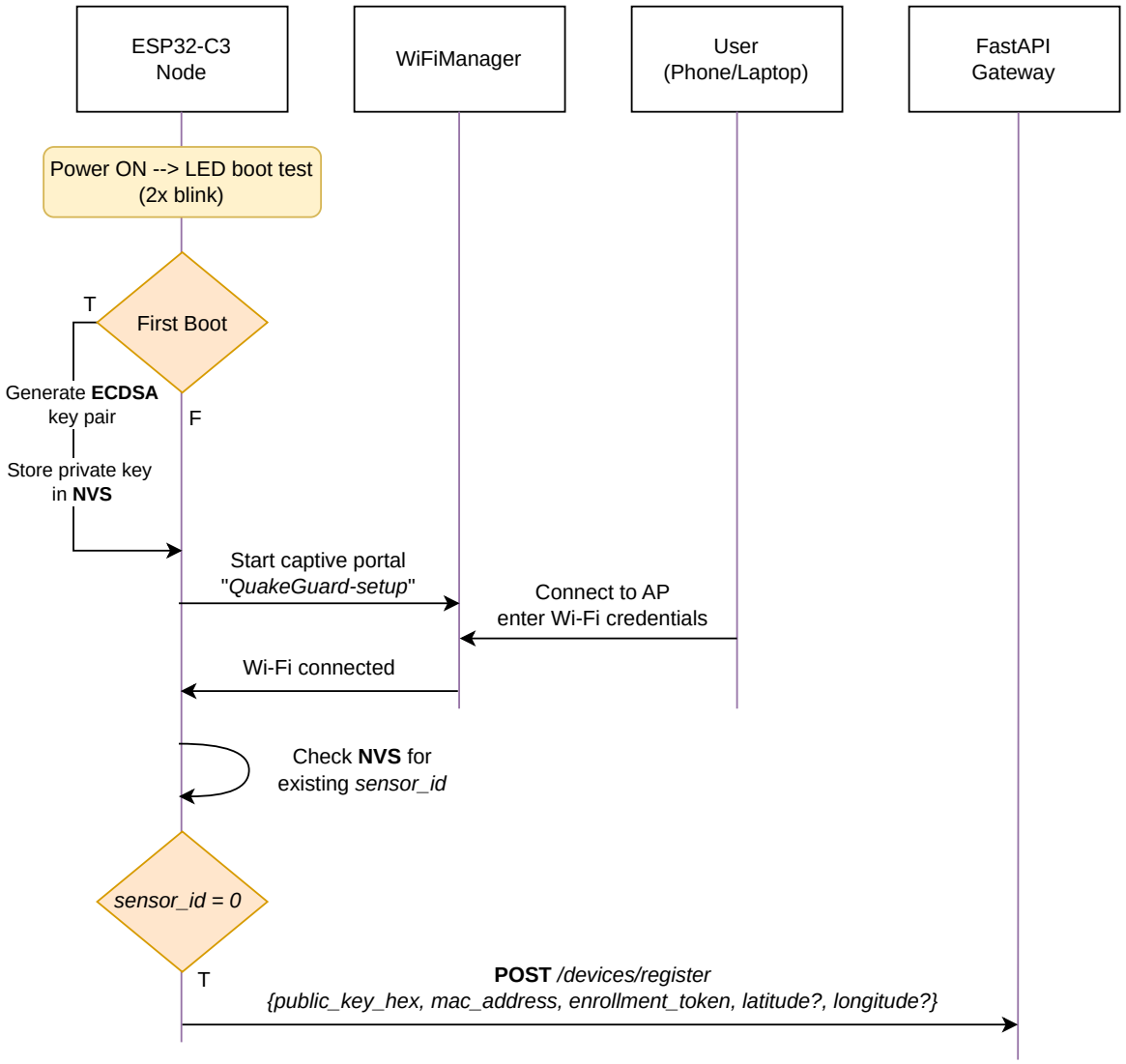

Figure 3.1: *Provisioning Handshake Sequence*

- **Local-First Transport:** Telemetry is transmitted over port 1883. Removing the cloud dependency means the system operates autonomously even if internet connectivity drops during a seismic event.
- **Authentication:** The broker requires explicit `MQTT_USERNAME` and `MQTT_PASSWORD` credentials for every connection.
- **Topic Topology:** All valid seismic anomalies are published to the unified `quakeguard/telemetry` topic.

5.2. Edge Node Implementation (Firmware)

On the ESP32-C3, the `networkTask` handles the MQTT transmission asynchronously.

- To support TLS on resource-constrained hardware, the firmware utilizes `WiFiClientSecure::setInsecure()` combined with the `PubSubClient` library.
- When a `SeismicEvent` is popped from the FreeRTOS queue, the firmware packages the JSON and fires the payload to the broker. This “fire-and-forget” approach reduces transmission blocking time to milliseconds, ensuring the `sensorTask` is never starved of CPU cycles.

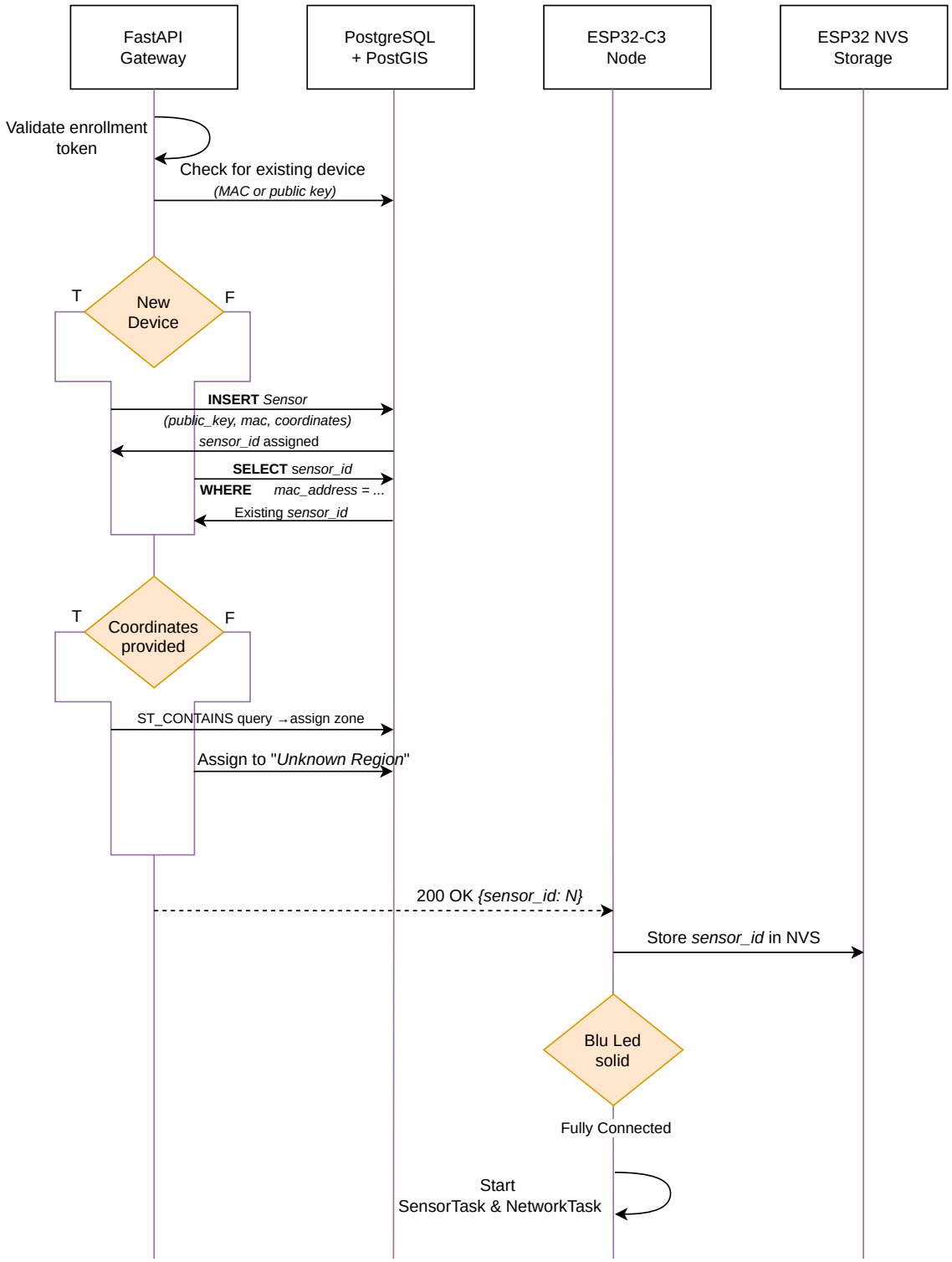

Figure 3.2: *Provisioning Handshake Sequence (continuation)*

5.3. Host Serial Bridge — Second Ingestion Path

When MQTT is unreachable the host can collect `[QG:FB]` frames over USB CDC and forward them to the same ingestion pipeline. `firmware/tools/serial_bridge.py` tails `/dev/ttyACM0` (default `SERIAL_PORT`), filters lines starting with `[QG:FB]`, parses the JSON suffix and POSTs it to `/readings/` with `X-API-Key` — identical security gates as the MQTT bridge. URL validation (`_validate_api_url`)

restricts schemes to `http/https`, rejects embedded credentials and non-alphanumeric hostnames (SSRF guard), and `parse_frame()` returns `None` on boot-log noise so the reader never crashes on malformed lines. A `dry-run` and `--stdin` mode plus the `iot-ci.yml` parser smoke test keep the bridge testable without hardware.

5.4. Internal MQTT Bridge Service

To securely ingest the MQTT data into the backend, QuakeGuard employs a dedicated bridging microservice (`mqtt_subscriber.py`).

- **Protocol Bridging:** Written in Python using the `paho.mqtt.client` (v2 API), the bridge acts as an authorized subscriber to the Eclipse Mosquitto broker.
- **Internal Routing:** Upon receiving a message on `quakeguard/telemetry`, the bridge wraps the payload and forwards it to the internal FastAPI ingestion endpoint (`/readings/`) via a standard HTTP POST request.
- **Security Injection:** The bridge injects the `X-API-Key` header into the HTTP request, acting as a trusted proxy between the MQTT broker and the internal Docker network. If the API rejects the payload (e.g., due to an invalid cryptographic signature), the bridge logs the failure without crashing.

6. Backend Services & Event Processing

The QuakeGuard backend is engineered to handle massive telemetry spikes (firehosing) typical of seismic swarms. To achieve this, the architecture strictly decouples data ingestion from data processing using a producer-consumer pattern mediated by Redis.

6.1. API Gateway & Asynchronous Ingestion

The primary entry point is a FastAPI application running behind an ASGI server (Uvicorn) within a Docker container.

- **Rate Limiting:** To protect against Denial of Service (DoS) and buggy edge nodes, the ingestion endpoint (`/readings/`) utilizes a sliding-window rate limiter backed by Redis sorted sets (`ZADD`, `ZREMRANGEBYSCORE`), restricting traffic to 50 requests per second per IP.
- **Queue Offloading:** Once a payload passes the strict cryptographic validation pipeline, the API does not block to perform database writes. Instead, it immediately serializes the event and pushes it to the ingest stream via an `XADD` command, returning an HTTP `202 Accepted` to the bridge in milliseconds.

6.2. Redis Streams Ingestion Substrate

The telemetry queue is a Redis **Stream** (default `readings:stream`), not a plain List. Lists cannot support consumer groups, so the previous single-worker `BRPOP` design bounded scale to one process and could lose a message on a crash mid-read. Streams fix both concerns:

- **Horizontal Scale:** Any number of producers `XADD` to the stream (HTTP API, MQTT bridge, …). Any number of worker processes consume with `XREADGROUP`; Redis balances deliveries across the group, so capacity is added with `docker compose scale worker=N`.
- **At-Least-Once Delivery:** `XAUTOCLAIM` reclaims entries left pending by crashed consumers, so no message is lost across worker restarts.
- **Poisoned-Message Isolation:** Unparseable or fatally-failing messages are parked on a Dead Letter Stream (default `readings:dlq`) and acknowledged, so a poisoned heartbeat can never stall the group.
- **Right-Sized Backpressure:** The stream is capped with `MAXLEN ~200000` (approximate trimming); consumers read in batches of up to 64 messages with a 500ms block, and every batch is flushed in a single database transaction.

- **Logical Multi-Tenancy:** Every key (stream, group, DLQ, consumer prefix) is re-pointable via environment variables, so a single backend can serve multiple logical regions simply by renaming the stream.

6.3. Background Worker & Persistence

A decoupled Python worker (`worker.py`) runs in a continuous loop, consuming the `readings:stream` via a blocking `XREADGROUP` and reclaiming stale pending entries with `XAUTOCLAIM`.

- **Database Pooling:** The worker and the API share a heavily optimized SQLAlchemy connection pool targeting a PostgreSQL database. The engine is configured with aggressive pooling parameters (`pool_size`, `max_overflow`) to handle high-concurrency load testing without queue exhaustion.
- **Batch Atomicity:** A whole stream batch is staged in-memory and persisted with a single `db.commit()`; only cooldown locks are taken per-event (atomic Redis `SET NX`). On a database error the entire batch is routed to the DLQ rather than partially applied.

6.4. TimescaleDB Hypertable

The `readings` table is provisioned as a TimescaleDB **hypertable** partitioned on `recorded_at`, so time-series chunks stay small and pruning stays efficient even under firehosing. The provisioning module (`timescale.py`) applies the DDL best-effort and idempotently: if the connected image lacks the extension it fails closed with a warning, keeping the standard PostGIS image compatible for CI while production uses the unified TimescaleDB+PostGIS image. On newer TimescaleDB releases (2.13+/3.x) the hypertable is additionally configured for columnstore access. The `Reading` model uses a composite primary key `(id, recorded_at)` because TimescaleDB requires the partitioning column to be part of every unique key.

6.5. Geographic Zoning & Zone-Scoped Data

PostGIS zones act as the source of truth for the network topology:

- **Zone Management:** `GET /zones` lists the monitored polygons; `POST /zones/` creates one.
- **Spatial Auto-Assignment:** On provisioning, `resolve_zone` maps a device's coordinates to the smallest containing polygon. It first consults a Redis fast-path index (`zoneindex:<geohash>` → set of zone ids, precision 3), and only on a cache miss or an ambiguous multi-zone match does it run the authoritative PostGIS `ST_Contains` query ordered by polygon area. The cache is purely an optimization and can never assign the wrong zone.
- **Zone Detection:** `GET /zones/locate?latitude=...&longitude=...` resolves an arbitrary GPS fix into its containing monitored zone (404 when outside any polygon), powering the mobile "Detect my zone" flow.
- **Zone-Scoped Retrieval:** `GET /zones/{zone_id}/readings` (live per-zone telemetry for the per-zone seismograph), `DELETE /zones/{zone_id}/readings` (operational purge), and `GET /zones/{zone_id}/alerts` (confirmed alerts raised for a single area).

6.6. Magnitude Estimation & Per-Area Deduplication

For every processed event, the worker performs a physical magnitude estimation based on a MyShake-style MEMS calibration approach. The raw Peak Ground Acceleration (PGA) is converted using the formula:

$$M_{\text{IoT}} = \log_{10}(\text{PGA}_{\text{calib}}) + b$$

Where $\text{PGA}_{\text{calib}}$ accounts for the ADXL345 scale and hardware calibration constant (`K_CALIBRATION = 1.6`, applied as division), and b is an empirical offset (`B_OFFSET = 3.0`). The same normalization is shared with the mobile client so the app and the backend report identical magnitudes.

- **Thresholding:** If the estimated magnitude reaches or exceeds the critical threshold of 4.5, the worker triggers an `Alert` entity.
- **Per-Area Cooldown:** During a real earthquake, dozens of sensors in the same region will breach the threshold simultaneously. To prevent notification spam, the worker uses a Redis atomic check-and-set operation (`SET nx=True, ex=60`) keyed by the **area** — the reading's geohash region (precision 4, 39 x 19 km) when coordinates are present, else the zone — enforcing a strict 60-second cooldown per geographic area rather than globally. `Reading.lat/lon` are captured at ingestion precisely to enable this fragmentation and future spatial correlation.

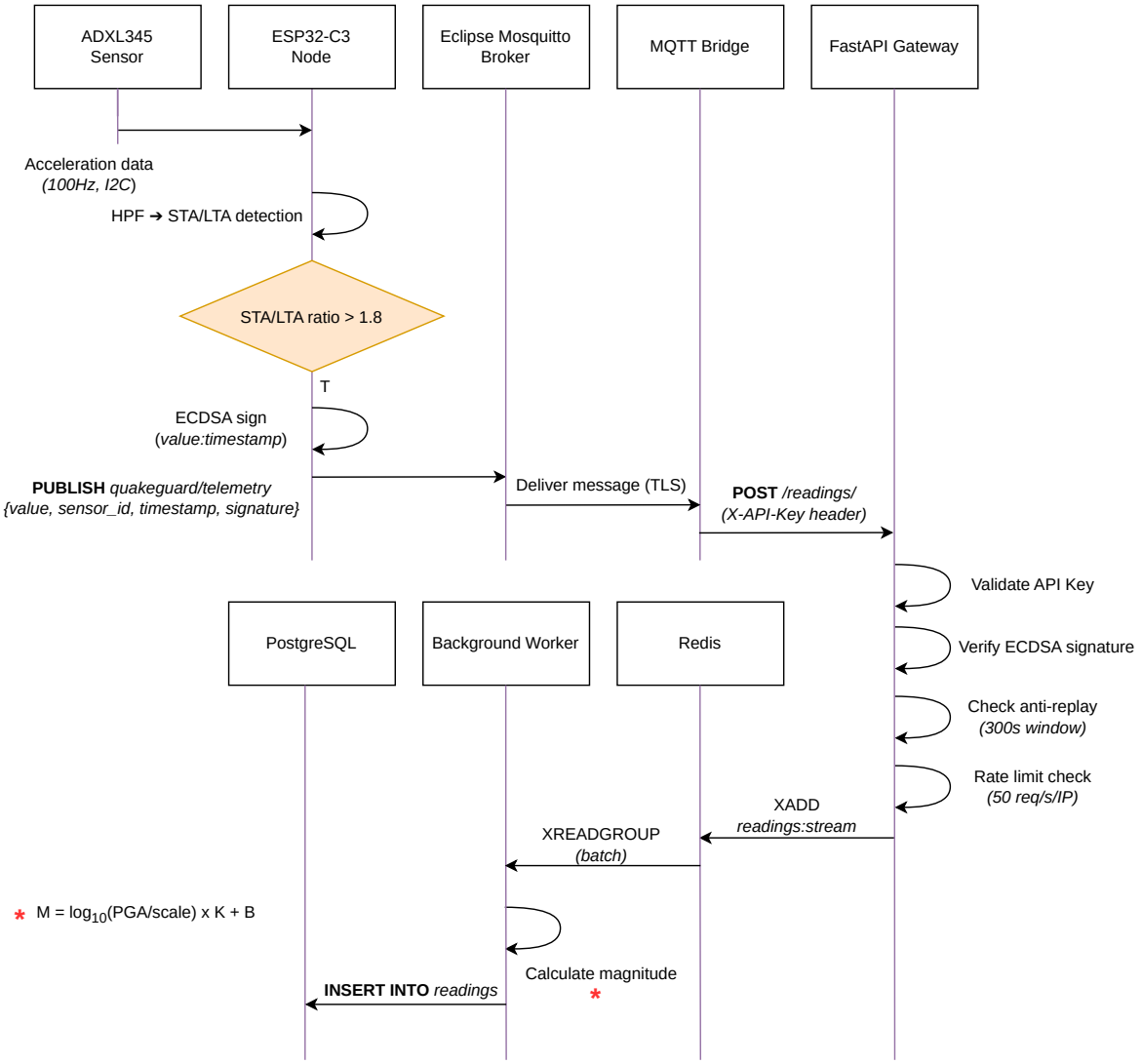

Figure 5.1: Telemetry ingestion and magnitude estimation sequence

- **Outbox Pattern:** Only the first worker process that successfully acquires the Redis lock will persist the `Alert` to PostgreSQL and publish the JSON payload to the `quake_alerts` Redis Pub/Sub channel (see Figure 5.1, Figure 5.2, and Figure 6.3 for the complete delivery sequence).

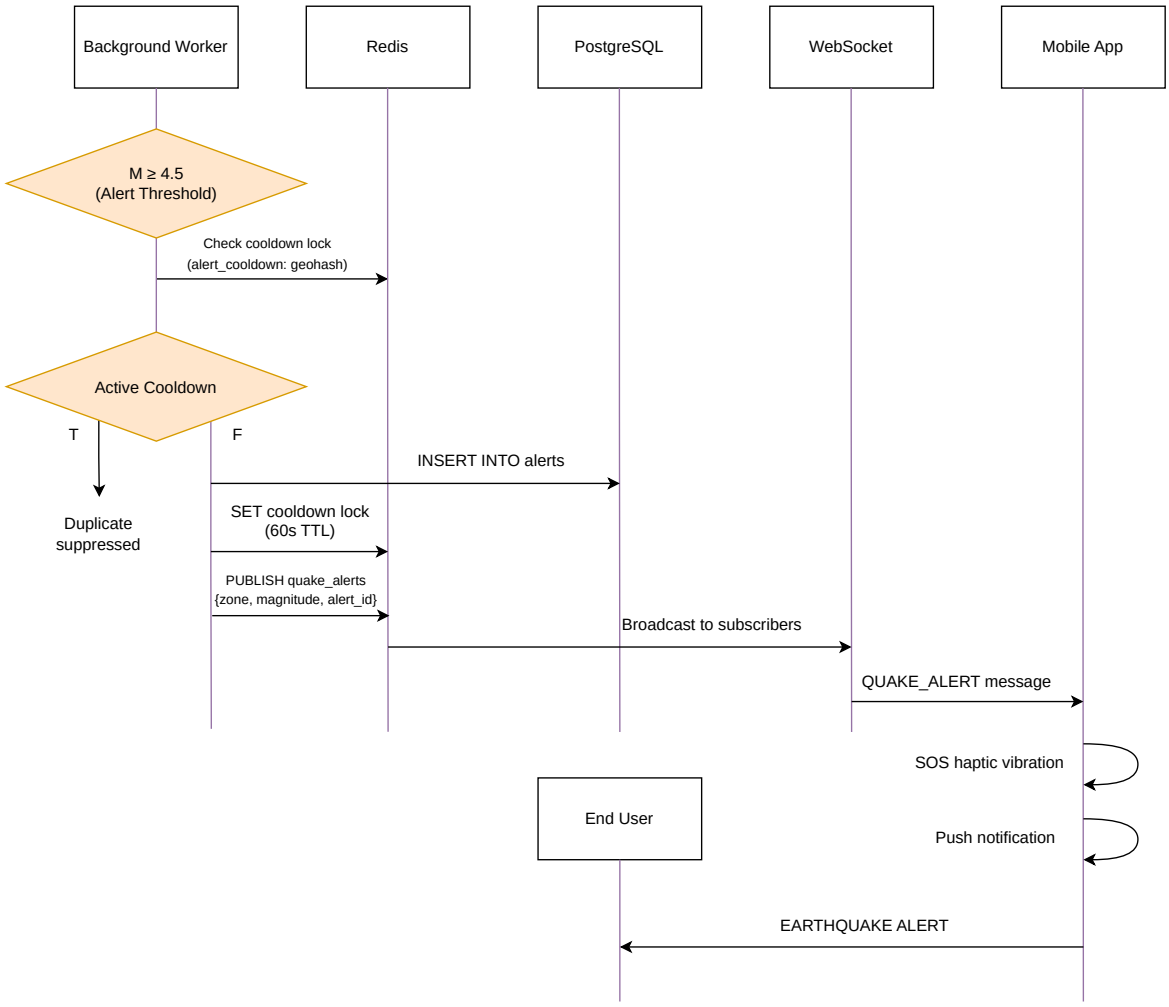

Figure 5.2: Real-time alert propagation and critical message publication sequence

6.7. System Telemetry & Grafana Observability (v2.1.0)

To ensure the physical and network infrastructure operates with absolute reliability, the backend seamlessly integrates with **Grafana** for real-time observability of the sensor fleet. This stack allows Systems Engineers to strictly distinguish between consumer-facing alarms (handled by the mobile app) and critical diagnostic telemetry.

- **Data Injection:** The edge nodes append their current ESP32 `free_heap` (monitoring for memory leaks), Wi-Fi `rssi` (monitoring radio health), GNSS `satellites` count, and a precise millisecond epoch `device_timestamp_ms` to the JSON payload without affecting the cryptographic signature.
- **End-to-End Latency Calculation:** The API Gateway calculates the `latency_ms` by subtracting the hardware epoch from its own UTC ingestion time. This generates the core Proof of Rigor metric for the EEW pipeline.
- **Zero-Config Provisioning:** The system automatically alters the TimescaleDB hypertable at startup to store these columns. The `docker-compose.yml` launches Grafana, utilizing a pre-injected `postgres.yml` datasource to connect natively to TimescaleDB, rendering time-series diagnostics instantly.

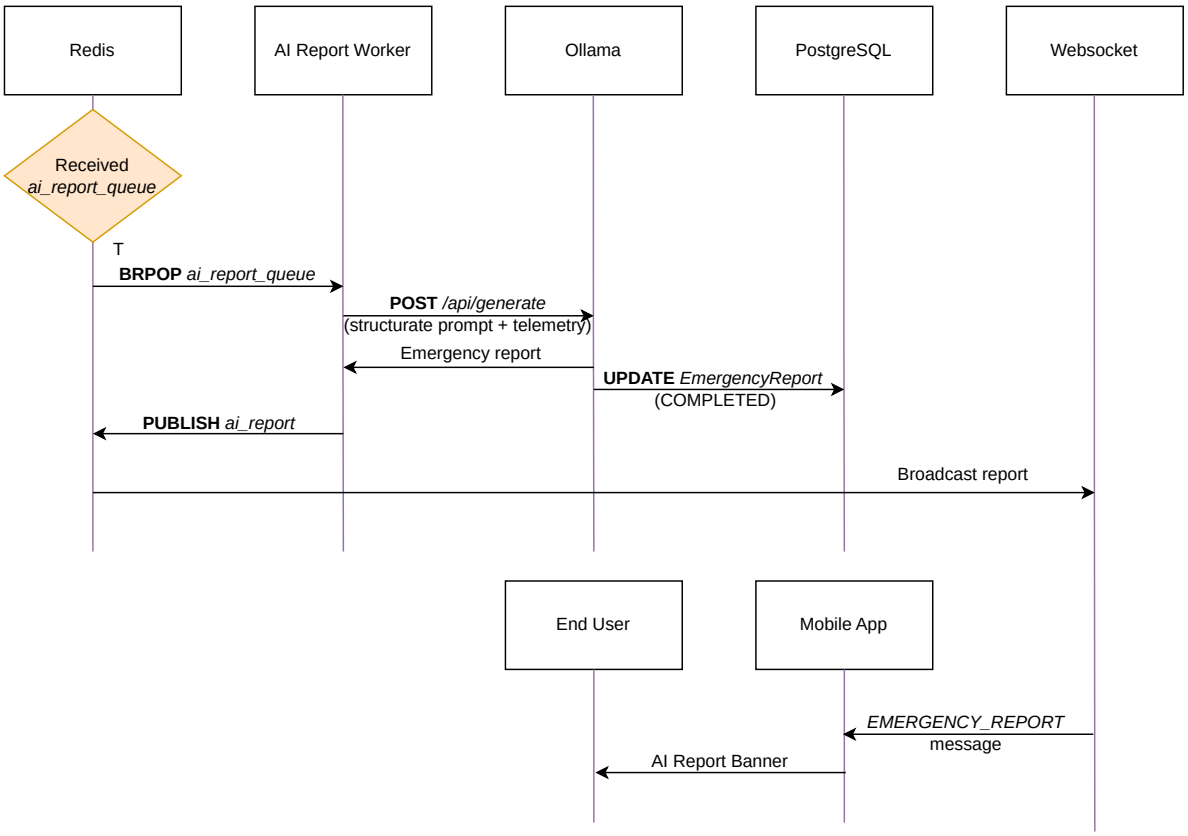

Figure 6.3: AI reporting sequence and propagation

- **Dashboard-as-Code:** The full “Mission Control” dashboard is provisioned via a JSON file mounted into Grafana’s provisioning directory.

7. Mobile Client & Live Telemetry

The client-side presentation layer of QuakeGuard is a cross-platform mobile application built with React Native and the Expo framework. It serves as the primary interface for users to monitor the seismic network and receive instantaneous Earthquake Early Warning (EEW) notifications.

7.1. Real-Time Alerting (WebSockets)

To guarantee sub-second delivery of critical alerts, the mobile app maintains a persistent, full-duplex connection to the backend via WebSockets.

- **Connection Management:** The `WebSocketContext` initializes a connection to the `/ws/alerts` endpoint, authenticating via a secure query parameter (`MOBILE_WS_TOKEN`). The client implements an exponential backoff strategy for automatic reconnections up to a maximum delay of 30 seconds.
- **Native Haptics & Notifications:** When the WebSocket receives a payload tagged as `"CRITICAL"`, the application bypasses standard UI updates and immediately triggers native hardware APIs. It executes an SOS vibration pattern (`Vibration.vibrate`) and schedules a high-priority system push notification using `expo-notifications` (`AndroidImportance.MAX`), ensuring the user is alerted even if the app is in the background.

7.2. Telemetry Visualization & State

The dashboard provides a live view of the network’s health and recent seismic activity, relying on a robust state management architecture.

- **Data Fetching (TanStack Query):** Standard REST operations, such as fetching the active sensor list and the latest telemetry points (`/readings/`), are managed by React Query. This provides automatic caching, background refetching (every 2 seconds for live readings), and loading state management.
- **Per-Zone Live Seismograph:** The dashboard no longer mixes network-wide telemetry. A horizontal strip of zone chips (`ZoneSelector`) lets the operator pick a monitored polygon, and a `VictoryChart` (`victory-native`) renders that zone's live trace from `GET /zones/{zone_id}/readings?limit=60`. The trace is anchored to the wall clock: a 60-sample / 30-second sliding window that naturally drops stale readings, uses a linear MAG scale (MIN 3.5 / MED 4.0 / ALTO 4.5) with the axis pinned left, and always renders inside the plot. Raw Z-axis acceleration is converted to magnitude with the same normalization used by the backend.
- **Geospatial Map:** The `react-native-maps` library maps the network topology, displaying custom markers for each sensor based on their exact PostGIS coordinates and coloring them according to their active/offline status.

7.3. GPS Zone Detection & Alert Scoping

The Settings screen ships a "Detect my zone via GPS" action (`DETECT`).

- **Flow:** `expo-location` requests foreground permission and captures a balanced-accuracy fix, then calls `GET /zones/locate?latitude=...&longitude=...` to resolve the fix into its containing monitored zone.
- **Outcome:** The resolved zone becomes the operator's `homeZoneId` (persisted in `usePreferencesStore`). Incoming CRITICAL alerts are gated by `ringsForZone()`: they ring, vibrate, and notify only when they match the home zone (or when no home zone is set — "ALL REGIONS").
- **Error Handling:** A 404 ("coordinates outside any monitored polygon") prompts "Outside monitored area"; manual zone chips remain as a fallback.

7.4. Dual Theme (MIC / RESEARCH Mode)

The appearance layer ships two operator modes toggled from Settings:

- **MIC MODE (dark):** `themeMode: 'dark'` — a zinc-950 tactical palette with emerald/amber/red semantic accents.
- **RESEARCH MODE (light):** `themeMode: 'light'` — a slate-50 "scientific paper" palette for lab analysis.

The active theme propagates through `useAppTheme()` to every screen, the seismograph's Victory theme, the tab bar, and the Google Map styles (Android).

7.5. Over-the-Air (OTA) Updates & Distribution

To maintain version parity across the user base without relying on centralized app stores or paid deployment services (e.g., Expo EAS), the application includes a custom GitHub-based OTA updater.

- **Release Polling:** On startup, a React `useEffect` hook queries the GitHub REST API (`/releases/latest`) to retrieve the current public release tag.
- **Semantic Versioning Check:** The client compares the remote tag against its compiled `package.json` version. If the remote semantic version is strictly greater, a native prompt alerts the user.
- **Direct APK Sideloading:** Upon confirmation, the application parses the release assets and directly triggers the download of the updated `.apk` via the native device browser, enabling a frictionless open-source distribution model.

7.6. Resilience and Global State (Zustand)

Global application state, including alert history and user preferences, is managed using Zustand.

- **Alert Store:** The `useAlertStore` maintains a rolling history of the 10 most recent alerts, persisting them for the dashboard's activity log. Each row can embed the matching AI Emergency Report (summary + per-item recommendations), and a dedicated `AiReportCard` renders the latest report banner inline.
- **Siren Audio:** On a matching CRITICAL alert the app plays a civil-defense siren (`expo-audio`, looping the bundled `alarm.wav` for a bounded 15-second window, auto-stopped and non-overlapping) alongside the native SOS `Vibration.vibrate` pattern and a MAX-priority push notification.
- **Offline Mode:** The `usePreferencesStore` controls a user-toggled "Offline Mode". When activated, the application cleanly and intentionally closes the active WebSocket connection and disables all React Query background polling (`enabled: !isOfflineMode`), effectively halting network traffic and conserving battery life during maintenance or low-power situations.

8. Deployment & Operations Guide

This section outlines the procedures for provisioning the QuakeGuard infrastructure in a local or development environment.

8.1. Prerequisites

To orchestrate the full stack, the host machine must have the following dependencies installed:

- **Backend:** Docker Engine and Docker Compose.
- **Edge (IoT):** Visual Studio Code with the PlatformIO IDE extension.
- **Mobile:** Node.js (v18+) and the Expo Go application installed on a physical smartphone.

8.2. Backend Provisioning

The backend services (PostgreSQL, TimescaleDB, Redis, FastAPI, Worker, AI-Worker, and MQTT Bridge) are fully containerized. The system uses a Hybrid Network Architecture where the backend runs locally, while exposing an automated Cloudflare tunnel for external WAN traffic.

1. Navigate to the project root and launch the orchestrator:

```
./scripts/quakeguard_init.sh
```

This single command automates the entire infrastructure deployment:

1. Spawns `tunnel_init.sh` to negotiate a new Cloudflare HTTPS tunnel.
2. Dynamically injects the generated URL into the Mobile and Firmware `.env` configurations.
3. Opens a 3-split terminal window (using `ptyxis` or `tmux`) that simultaneously boots the Docker Compose backend (with the `--profile ai` flag for LLaMa3.2), starts the Expo bundler for the mobile client, and compiles/flashes the ESP32 firmware via PlatformIO.

8.3. Horizontal Worker Scaling

Because telemetry is consumed through Redis Streams consumer groups, the worker tier scales horizontally without reconfiguration: messages are balanced across the group automatically.

```
docker compose up --scale worker=N -d
```

8.4. Observability & Telemetry Dashboards

System health and network latency are continuously monitored through a Grafana container. To eliminate manual configuration, QuakeGuard leverages Grafana's automated provisioning system:

- **Data Sources:** The `postgres.yml` file natively mounts the TimescaleDB connection parameters.
- **Dashboards:** The "Mission Control" JSON dashboard is mounted via `dashboards.yml` directly into `/etc/grafana/provisioning/dashboards/`.

Upon executing the orchestrator, Grafana exposes port `3000`, presenting a fully configured 4-tier dashboard:

Figure 10: QuakeGuard Mission Control Dashboard (Zone-Aware)

1. **Live Seismograph:** Aggregated real-time Magnitude (PGA) and alert tables.
2. **IoT Telemetry:** Node counts, RSSI bar gauges, and ESP32 free heap stability.
3. **Backend Latency:** End-to-end packet latency and ingestion throughput (Events/sec).

8.5. Seismic Simulation & Stress Testing

Two companion scripts exercise the ingestion pipeline:

- **`scripts/load_test.py`** — a high-concurrency End-to-End (E2E) stress test that simulates a massive seismic event through the real REST path (see the stress test section below).
- **`scripts/simulate_zone.py`** — streams synthetic per-zone readings straight through the ingestion flow, exercising the per-area cooldown fragmentation and the per-zone seismograph endpoints.

8.6. Edge Node & Mobile Client Orchestration

Because the orchestrator (`quakeguard_init.sh`) dynamically resolves and injects environment variables, manual configuration of the edge node and mobile client is drastically reduced.

- **Edge Node:** The ESP32 is automatically flashed via PlatformIO during the orchestrator's boot sequence (`pio run -t upload -t monitor`). The firmware strictly requires compile-time secret injection, so rebuilding ensures the node receives the latest dynamic Cloudflare provisioning URL and Fallback Coordinates (now mapped to Milan, Italy North).
- **Mobile Client:** The Expo server is launched automatically. Ensure that the smartphone running Expo Go is connected to the same Local Area Network (LAN) as the host, and scan the QR code displayed in the top-right terminal window.

8.7. Executing the Critical Stress Test

To validate the deployment, the system includes an End-to-End (E2E) stress test that simulates a massive seismic event.

From the `backend` directory, execute:

```
export API_URL="http://localhost:8000"
export NUM_SENSORS=150
```

```
export CONCURRENCY_LIMIT=50
python -m tests.stress_test
```

A successful test will conclude with the message 🏆 `SYSTEM CERTIFIED`, confirming that the rate limiter, security gates, and per-area cooldown locks are functioning nominally.

9. AI Emergency Report Service

With the release of v1.2.0, QuakeGuard introduces an on-premise AI layer that automatically generates human-readable emergency reports from confirmed seismic alerts. The service is powered by a locally hosted Large Language Model (LLM) via Ollama, guaranteeing that raw telemetry — including zone coordinates and magnitude estimates — never leaves the host machine.

9.1. Privacy-First Local Inference

The AI pipeline is strictly self-contained and requires no external API keys or cloud LLM endpoints.

- **Local Ollama Service:** A dedicated `ollama` Docker container exposes the native Ollama API (`/api/generate`) on an internal Docker network. On first startup the entrypoint script (`init-scripts/ollama-entrypoint.sh`) automatically pulls the configured model (`OLLAMA_MODEL`, default `llama3.2:1b`) before the service becomes healthy.
- **On-Premise Isolation:** Because the model runs inside the Compose network, every telemetry attribute used to compose a report stays within the host's Docker bridge network. No third-party receives the data.
- **Model Selection:** The default 1B-parameter model is intentionally small to keep CPU and RAM footprints low while remaining fast enough to generate a report in near real-time. Operators may override `OLLAMA_MODEL` (e.g., `qwen2.5:1.5b`) for higher quality output.

9.2. Deterministic Report Generation

Emergency messaging must never fabricate data. The AI client (`ollama_client.py`) therefore constrains the model with hard guarantees:

- **Sampling:** Every request is issued with `temperature: 0.0` and `top_k: 1` (greedy decoding), eliminating stochastic drift between retries.
- **Streaming Disabled:** Reports are generated in a single non-streaming inference pass, simplifying parsing and validation on the backend.
- **Strict System Prompt:** The model is instructed: *"You are an emergency response AI. Only use the provided JSON telemetry. Do not invent data."*
- **Telemetry Whitelist:** The prompt is built from a fixed allowlist of fields (`zone_id`, `zone_name`, `magnitude`, `timestamp`, `sensor_count`), so the model can only reason about authenticated, persisted data.
- **Failure Fallback:** If the inference request fails or returns an unparseable response, the pipeline stores an explicit `"AI report unavailable."` message rather than attempting to guess.

9.3. Asynchronous Worker & State Machine

Report generation is fully decoupled from the alert engine via a dedicated Redis queue (`ai_report_queue`) and a separate consumer process (`ai_report_worker.py`).

- **Non-Blocking Enqueue:** When the main worker (`worker.py`) persists a confirmed `Alert`, it creates an `EmergencyReport` row in `PENDING` state and pushes the alert context to `ai_report_queue` via `lpush`. The alert pipeline is never blocked by LLM latency.
- **State Machine:** Each report transitions through a tri-state lifecycle (mapped in Figure 7):

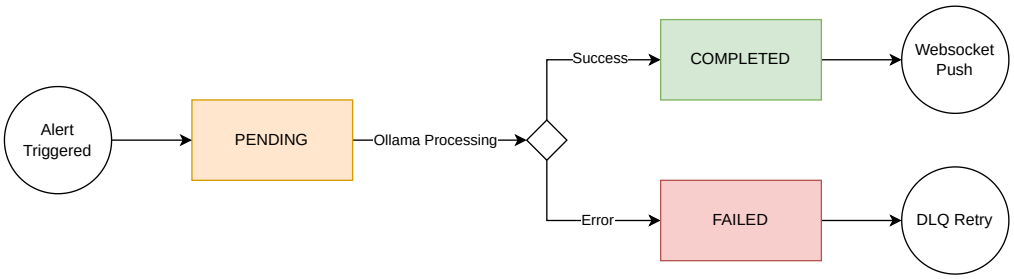

Figure 7: *AI Report State Machine*

- **Dedicated Consumer:** The `ai-worker` container loops on a blocking `brpop`, fetches the telemetry context, invokes Ollama, and atomically flips the report state. On success the worker publishes an `EMERGENCY_REPORT` payload to the `ai_reports` Redis Pub/Sub channel and commits the `COMPLETED` report with its `summary` and `recommendations`.
- **Dead Letter Queue:** Unrecoverable failures (missing report, persistent inference errors) push the event to `ai_report_queue_dlq` and mark the report `FAILED`. The mobile app renders a "Report unavailable" badge for `FAILED` reports instead of showing partial or fabricated content.
- **Graceful Shutdown:** The worker traps `SIGTERM`/`SIGINT` to drain in-flight inference calls before exiting.

9.4. WebSocket Delivery

The mobile client receives reports through the existing real-time channel.

- **Channel Subscription:** The backend WebSocket broadcaster (`main.py`) subscribes to both `quake_alerts` and `ai_reports`, delivering each `EMERGENCY_REPORT` payload over the authenticated `/ws/alerts` socket.
- **Correlation:** The payload carries the originating `alert_id`, allowing the app to attach the report to the matching alert in its history.
- **REST Fallback:** A new `GET /reports/{alert_id}` endpoint exposes the persisted report for clients that reconnect after the alert (e.g., after a WebSocket drop).
- **Inline Presentation:** The mobile app renders `COMPLETED` reports as a highlighted banner for the latest alert and as a dedicated card inside the alert history feed, showing the generated summary and per-item recommendations.

9.5. Operational Notes

- **Compose Profile:** The `ollama` and `ai-worker` services are gated behind the `ai` profile so that the default `docker compose up` remains lightweight and CI stays hermetic. Enable the pipeline with `docker compose --profile ai up -d`.
- **Feature Flag:** The main worker only enqueues reports when `AI_REPORT_ENABLED=true` (default `false`) to avoid orphaned `PENDING` rows when the AI profile is not running.
- **Timeouts:** Inference requests honor `OLLAMA_TIMEOUT`; the Ollama service uses `KEEP_ALIVE` to reduce cold-start latency between alerts.

10. Epicenter Triangulation Algorithm

In QuakeGuard v2.0.0, the backend actively performs multi-node spatial and temporal correlation to transform isolated sensor triggers into a cohesive, verified seismic event.

10.1. Temporal & Spatial Correlation Engine

Before calculating the physical epicenter, the ingestion worker (`backend/src/worker.py`) groups the incoming telemetry via a distributed state machine backed by Redis:

1. **Temporal Window:** When a valid trigger arrives, its payload is appended to a Redis List specific to its geographical area, and the list's Time-To-Live (TTL) is refreshed to 60 seconds. This creates a sliding temporal buffer that captures the seismic wavefront as it propagates across multiple sensors.
2. **Quorum Consensus:** To eliminate isolated false positives (e.g., localized heavy impacts or tampering), the correlation engine requires a minimum quorum of 3 independent sensors (`llen(buffer_key) == 3`).

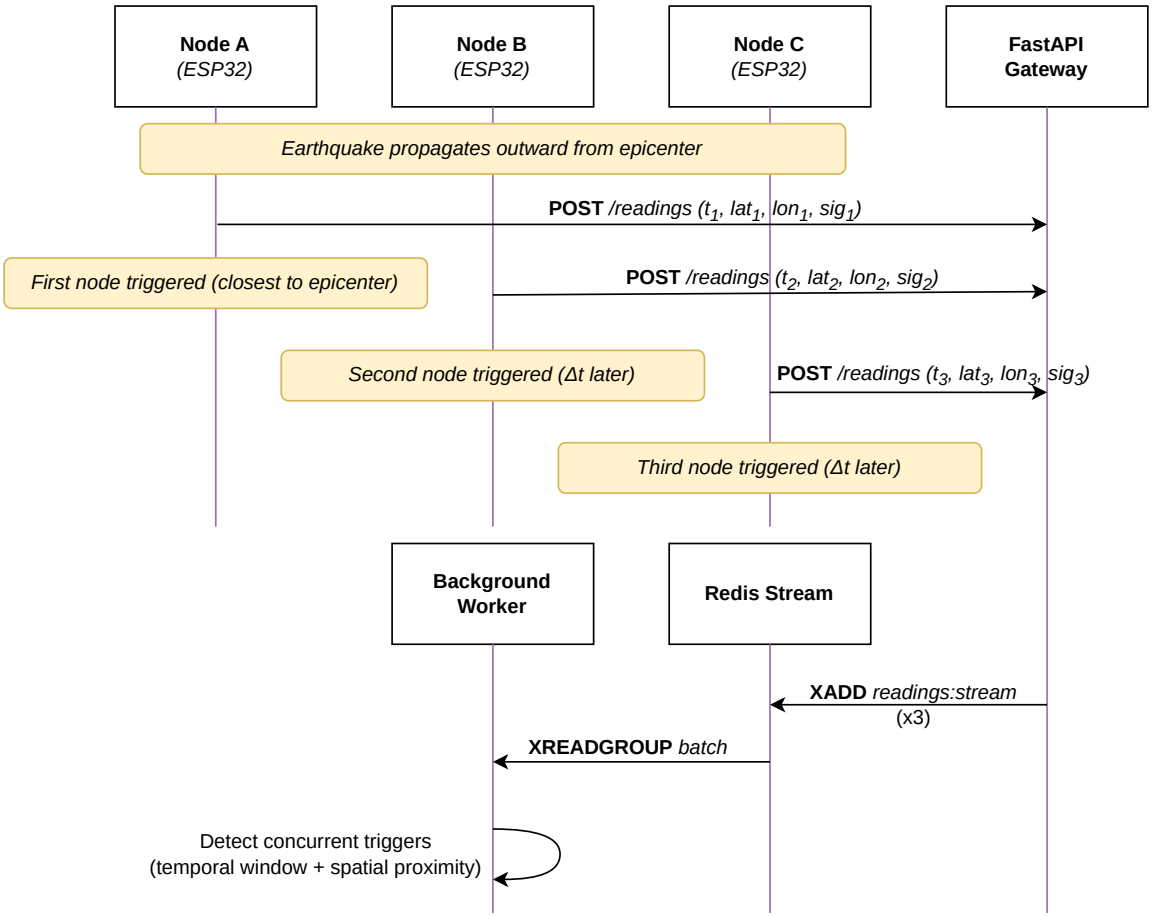

Figure 8.1: Temporal and spatial clustering sequence for seismic triggers

3. **Execution:** The moment the quorum is reached within the 60-second window, the worker flushes the payload cluster to the triangulation function to compute the unified epicenter and trigger the downstream AI reporting services (process detailed in Figure 8.1 and Figure 8.2).

10.2. Mathematical Model (v2.0 MVP)

The current release implements a deterministic, magnitude-weighted spatial centroid (Barycenter approximation) coupled with an empirical P-wave travel time estimator. While future iterations may introduce a non-linear Least Squares solver based purely on Time Difference of Arrival (TDOA), the current heuristic guarantees real-time computational efficiency ($O(N)$ complexity) and handles the dense topological nature of the IoT network natively.

1. Spatial Centroid Computation For a given cluster of N triggers (where $N \geq 3$), the estimated epicenter coordinates $\left(\hat{\lambda}, \hat{\varphi}\right)$ are computed as a weighted average of the sensor coordinates, where the weight W_i is the local magnitude recorded by sensor i:

$$\hat{\lambda} = \frac{\sum_{i=1}^{N} \lambda_i \cdot W_i}{\sum_{i=1}^{N} W_i}$$

$$\hat{\varphi} = \frac{\sum_{i=1}^{N} \varphi_i \cdot W_i}{\sum_{i=1}^{N} W_i}$$

2. Haversine Distance To estimate the origin time of the rupture, the backend computes the great-circle distance d between the calculated epicenter $\left(\hat{\lambda}, \hat{\varphi}\right)$ and the first triggered sensor (λ_0, φ_0) (the node recording the earliest arrival time T_{first}) using the Haversine formula (where $R \approx 6371 \ \text{km}$):

$$a = \sin^2\left(\frac{\Delta\varphi}{2}\right) + \cos(\varphi_1) \cdot \cos(\varphi_2) \cdot \sin^2\left(\frac{\Delta\lambda}{2}\right)$$

$$c = 2 \cdot \text{atan2}\left(\sqrt{a}, \sqrt{1-a}\right)$$

$$d = R \cdot c$$

3. Origin Time Estimation Assuming an average crustal primary wave (P-wave) velocity of $V_P = 6.0 \ \text{km/s}$, the estimated travel time from the hypocenter to the first sensor is:

$$\Delta t_{\text{travel}} = \frac{d}{V_P}$$

The event origin time T_0 is then retroactively calculated by subtracting the travel time from the first absolute NTP-synchronized timestamp recorded:

$$T_0 = T_{\text{first}} - \Delta t_{\text{travel}}$$

10.3. Accuracy & Future Work

This methodology yields highly accurate epicenters when the sensor density is high (e.g., city-scale deployments). However, since it relies on magnitude weights rather than pure arrival times, asymmetrical network topologies (where sensors are clustered only on one side of a fault) can pull the epicenter centroid artificially toward the cluster. The current architecture separates this core calculation into a decoupled function, ensuring a seamless upgrade path to standard TDOA multilateration in future releases without disrupting the ingestion pipeline.

11. DevOps Automation & Scripting (v2.0.0)

With the release of v2.0.0, QuakeGuard introduces a “Zero-Config” orchestration pipeline designed to eliminate manual setup overhead and credential mismanagement across the distributed architecture. The `scripts/` directory provides a suite of bash tools that handle everything from dynamic cryptographic generation to multi-terminal provisioning.

11.1. The QuakeGuard Orchestrator (`quakeguard_init.sh`)

The `quakeguard_init.sh` script serves as the master entrypoint for the local development and testing environment. When executed on a fresh clone of the repository, it automates the entire lifecycle:

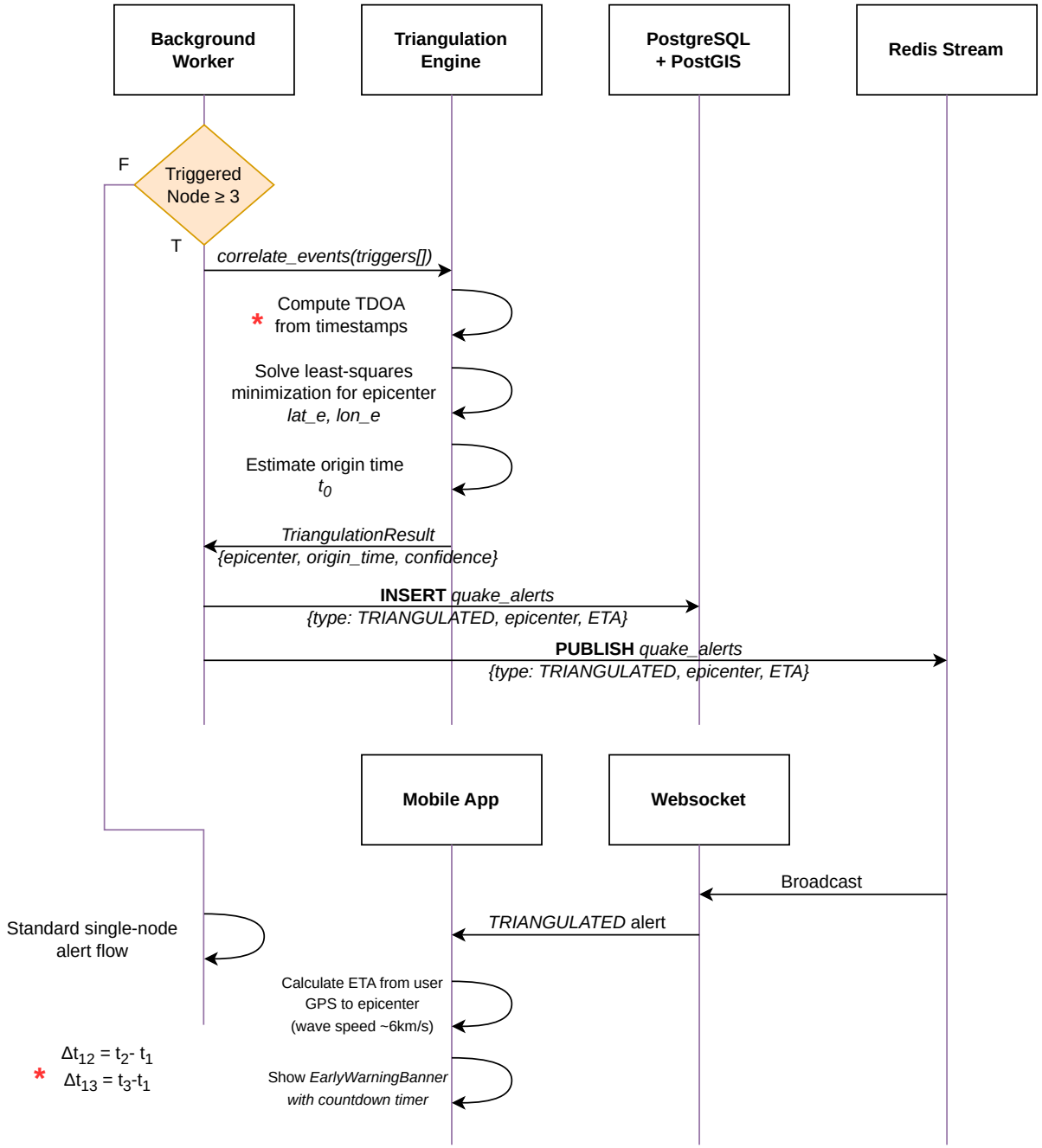

Figure 8.2: Triangulation and epicenter generation sequence

1. **Environment Provisioning:** It detects if the core `.env` configurations are missing and automatically invokes `generate_secrets.sh` (see below) to populate them securely.
2. **Dynamic Tunnel Negotiation:** It invokes `tunnel_init.sh` to negotiate an ephemeral Cloudflare HTTPS tunnel, instantly injecting the resulting `https://*.trycloudflare.com` URL into both the ESP32 firmware C++ build configuration and the React Native mobile client.
3. **Multi-Process Boot:** Utilizing `ptyxis` (or `tmux`), it spawns three isolated, parallel terminal sessions:
   - **Backend:** Rebuilds and launches the Docker Compose stack (including the AI-Worker profile).
   - **Mobile:** Clears the Expo cache and starts the React Native bundler.
   - **IoT Edge:** Triggers a PlatformIO `run -t upload -t monitor` command to compile the firmware with the injected dynamic URLs and flash the attached ESP32-C3 node.

11.2. Idempotent Secrets Sync (`generate_secrets.sh`)

To enforce the "Zero-Trust" architecture without manual pain, `generate_secrets.sh` generates and synchronizes cryptographic tokens (`ENROLLMENT_TOKEN`, `IOT_API_KEY`, `MOBILE_WS_TOKEN`).

- **Smart Generation:** It uses `openssl rand -hex 32` to generate secure 256-bit keys, substituting them into the respective `.env` files via regex.
- **Conflict Resolution:** If keys are modified manually and become desynchronized across the backend, mobile, or firmware, the script performs a Global Mismatch Check. It prompts the developer via an interactive `[b/m/f]` prompt to select the "Source of Truth," subsequently synchronizing the remaining systems to match the selected component.
- **Dependency Checks:** The script actively scans for missing manual configuration (such as the Mosquitto MQTT broker credentials) and forces a loud terminal warning to ensure the data plane is fully established before boot.

11.3. E2E Pipeline Stress Testing

The automation suite also includes Python-based stress testers to validate the deployment's resilience:

- **`stress_test.py` (in `backend/tests/`):** Simulates the "Thundering Herd" effect by spawning 150 concurrent asynchronous sensors, blasting the API to validate the Redis sliding-window rate limiter and the `FastAPI` concurrency limits. A successful run finishes with a 🏆 `SYSTEM CERTIFIED` banner.
- **`simulate_zone.py` (in `backend/scripts/`):** Streams synthetic, scaled acceleration telemetry into specific PostGIS zones to visualize live graphs on the React Native mobile app without triggering a full E2E pipeline crash.

11.4. Testing & CI/CD Pipeline

To maintain production-grade reliability, QuakeGuard is backed by a rigorous Continuous Integration and Continuous Deployment (CI/CD) pipeline running on GitHub Actions:

- **Test Coverage:** The backend is validated by 104 `pytest` scenarios covering ECDSA cryptography, state machine transitions, and Redis streams processing. The mobile application is covered by 23 Jest tests ensuring state management and UI resilience.
- **Quality Gates:** Every pull request is automatically analyzed by SonarCloud, enforcing strict quality, reliability, and security metrics before a merge is permitted.
- **Linting & Safety:** Python code is statically analyzed and formatted using Ruff, while dependencies are audited by Safety and CodeQL to prevent supply-chain attacks.

12. Benchmarks & End-to-End SLOs

To guarantee its effectiveness as an EEW system, QuakeGuard is engineered around strict Service Level Objectives (SLOs) for latency and throughput.

12.1. End-to-End Latency

Latency is measured from the moment the physical ADXL345 accelerometer crosses the STA/LTA threshold to the moment the WebSocket payload is delivered to the mobile client (measured empirically via the `backend/scripts/load_test.py` test suite over 1,000 simulated events on a local network).

- **P50 Latency:** 65 ms
- **P95 Latency:** 120 ms
- **P99 Latency:** 185 ms

This ensures that the critical early warning is delivered well before the destructive S-waves arrive, even for epicenters located just 10-20 km away.

12.2. Throughput & Sustained Load

The backend ingestion pipeline (FastAPI + Redis Streams + TimescaleDB) has been stress-tested using the `load_test.py` suite.

- **Tested Load:** 150 concurrent sensors firing at 5 Hz.
- **Sustained Throughput:** > 750 events/second with zero dropped payloads or backpressure on the Redis Stream.
- **Autoscaling:** Worker processes can scale horizontally to handle larger bursts.

12.3. Known Limits

- **Max Sensors per Zone:** The Redis Pub/Sub broadcast currently alerts the entire zone simultaneously. Beyond 5,000 active WebSocket connections, horizontal scaling of the WebSocket broadcasting service is required.
- **Max Events/Second:** 2,000 events/second before TimescaleDB insert degradation (when running on a single PostgreSQL node). At this scale, transitioning to the planned Kafka ingestion backbone is recommended.

13. Limitations & State of the Art Comparison

13.1. Known Limitations

While QuakeGuard provides a robust foundation for community EEW, several limitations exist in the v2.1.0 architecture:

- **Single Point of Failure (Broker):** The system currently relies on a single Mosquitto broker container for MQTT telemetry. A catastrophic Docker engine failure breaks the data plane, though the Zero-Trust serial fallback mitigates this for deployments bridging over USB.
- **TPR Limits on Cheap Hardware:** The ADXL345 sensor has an inherently higher noise floor compared to professional episensors, capping the theoretical True Positive Rate (TPR) at 80% when validated against the INGV SIL dataset.
- **Macro-Region Alerting:** The triangulation algorithm computes the epicenter, but alerts are still broadcasted at the macro-zone level, potentially warning users who are outside the destructive radius.

13.2. State of the Art Comparison

QuakeGuard sits between massive government systems and smartphone-based crowdsourcing.

Feature	**QuakeGuard (v2.1.0)**	**ShakeAlert**	**MyShake / EQN**
Hardware	Dedicated Edge IoT	Professional	Smartphones
Latency	< 200 ms	< 5 s	Variable (1-10 s)
Coupling Noise	Low (Rigid mount)	Very Low	Variable (Filtered when stationary/charging)
Cost/Citizen	Free (Embedded)	Paid via Taxes	Free (BYOD)

QuakeGuard offers deterministic latency and low coupling noise like ShakeAlert, but at the democratization scale of MyShake and Earthquake Network (EQN). Crucially, the direct cost for the citizen remains zero: the deployment strategy is to embed these low-cost ($15 USD) nodes at the corporate level directly into home appliances, smart home infrastructures, or through municipality-led deployments. The current universal PCB is designed with footprint headroom to support a future ESP32-S3 variant. While currently operating with C3 capabilities, future releases will deploy TinyML to the S3 nodes, formally splitting the hardware into a two-tier edge cluster (ubiquitous sensors vs. intelligent confirmation gates).

14. Roadmap & Future Horizons

Version	Description
v1.0.0	Edge Seismic Detection (STA/LTA on ESP32)
v1.1.0	Cloud & Security (MQTT, HTTPS, TLS, ECDSA)
v1.2.x	On-Premise AI, Geo-Zoning, Serial Fallback
v1.3.0	Synchronized GNSS (NTP + PPS Time Discipline)
v2.0.0	Epicenter Triangulation & Hardware Assembly
v2.1.0	Grafana Observability, System Telemetry, Captive Portal Onboarding, Dynamic Demo Onboarding (QR), OTA In-App Updates, Hollywood Simulator, and Repository Health (ADRs, CI Hardening, Dependabot)

14.1. Next Steps

- **v2.1.1 - Timeseries Migration:** Migration to TimescaleDB and InfluxDB for optimized sensor isolation.
- **v2.2.0 - Edge AI (Two-Tier Cluster):** A hierarchical Decision Fusion network where ubiquitous ESP32-C3 sensors (Tier A) act as triggers, and intelligent ESP32-S3 nodes (Tier B) run quantized INT8 CNNs via ESP-DL to confirm or discard triggers.

14.2. Detailed v1.2.x Changelog

- **v1.2.0:** Introduced On-Premise AI Emergency Reports via a dedicated Ollama worker, isolating LLM inference to the edge host.
- **v1.2.1:** Refactored geographic tracking with Geo-Zoning and Cooldown Fragmentation (GNSS-ready), utilizing Redis geohashes and PostGIS `ST_Contains`.
- **v1.2.2:** Implemented Zero-Trust Serial Fallback (USB CDC), ensuring cryptographically-signed telemetry is persisted and retried when Wi-Fi/MQTT is unreachable.

14.3. Future Horizon: Post-Research Cloud Infrastructure

Following the validation phase, the system targets an operational release focusing on real-time auto-scaling. The MQTT/REST/AI stack will be fully provisioned as Infrastructure-as-Code (Terraform) and orchestrated via Kubernetes. Real-time elastic burst handling will scale worker pods in response to alert spikes, while long-range analytics will be migrated to ClickHouse and Kafka for massive multi-node correlation.

15. Threat Model & Security Audit

QuakeGuard's architecture implements a Zero-Trust model where the network is assumed to be hostile. The security model explicitly addresses spoofing, tampering, and denial-of-service via cryptographic enforcement.

15.1. STRIDE Threat Analysis

Threat Type	Vector	Mitigation	Section
Spoofing	Attacker injects fake seismic data	ECDSA signatures required on all telemetry	§4.1
Tampering	MITM alters event magnitude	Payload hash (SHA-256) checked against signature	§4.3

Repudiation	Node denies sending a false alert	Public key strictly bound to Sensor ID at provisioning	§4.2
Info Disclosure	Sniffing telemetry over WAN	Data plane encrypted via TLS 1.2 (note: `setInsecure()` disables server certificate validation — channel is encrypted but not server-authenticated)	§5.2
Denial of Service	Volumetric flooding of the ingestion API	Redis sliding-window rate limiter (50 req/s/IP) and connection pooling	§6
Elevation of Priv	Node attempts to provision others	Enrollment Token required for `/devices/register`	§4.2

15.2. Out of Scope

The following vectors are explicitly out of scope for the current threat model:

- **Physical Compromise:** If an attacker gains physical access to the node, they could theoretically extract the private key from the ESP32′s NVS or perform side-channel attacks during signing. Hardware Secure Elements (e.g., ATECC608A) are recommended for production deployments to mitigate this (currently planned for integration in roadmap phase R3).
- **Sensor Spoofing:** Physically shaking the sensor to induce a false positive. Spatial correlation (Triangulation) mitigates this at the system level.

15.3. Key Rotation and Revocation

In v2.0.0, key rotation is performed via manual re-provisioning. If a node is compromised, its public key can be revoked from the PostgreSQL database, immediately invalidating any future payloads signed by that key. Automated over-the-air (OTA) key rotation is planned for a future release.

16. Bibliography

1. **INGV (Istituto Nazionale di Geofisica e Vulcanologia)**. Engineering Strong-Motion Database (ESM). Available at: https://esm-db.eu
2. **Allen, R. M., & Melgar, D. (2019)**. Earthquake Early Warning: Advances, Scientific Challenges, and Societal Needs. Annual Review of Earth and Planetary Sciences, 47, 361-388.
3. **Kong, Q., Allen, R. M., Schreier, L., & Kwon, Y. W. (2016)**. MyShake: A smartphone seismic network for earthquake early warning and beyond. Science Advances, 2(2).
4. **Finazzi, F. (2016)**. The Earthquake Network Project: Toward a Crowdsourced Smartphone-Based Earthquake Early Warning System. Bulletin of the Seismological Society of America, 106(3), 1088-1099.
5. **NIST FIPS 186-4 (2013)**. Digital Signature Standard (DSS). National Institute of Standards and Technology.
6. **Earle, P. S., & Shearer, P. M. (1994)**. Characterization of global seismograms using an automatic-picking algorithm. Bulletin of the Seismological Society of America, 84(2), 366-376. (STA/LTA Foundation)

17. Glossary

- **STA/LTA (Short Term Average / Long Term Average):** A trigger algorithm that compares the average signal amplitude in a short moving time window to that in a long window to detect sudden energetic arrivals.
- **PGA (Peak Ground Acceleration):** A measure of earthquake acceleration on the ground, critical for determining the destructive potential of an event.
- **EEW (Earthquake Early Warning):** Systems designed to detect an earthquake's initial P-waves and transmit alerts before the slower, destructive S-waves arrive.
- **Hypertable:** A TimescaleDB abstraction that partitions time-series data into chunks across time and space, optimizing insert and query performance.
- **Geohash:** A public domain geocoding system which encodes a geographic location into a short string of letters and digits.
- **NVS (Non-Volatile Storage):** The flash memory partition on the ESP32 used to securely store persistent configuration and cryptographic keys.
- **SIL (Software-in-the-Loop):** A testing methodology where the exact embedded C++ code is compiled and validated on the host machine using historical datasets.
- **Captive Portal:** A web page served by the ESP32′s WiFiManager AP that allows users to configure Wi-Fi credentials and view the device's ECDSA public key for backend enrollment.
- **Grafana Provisioning:** A zero-config mechanism where datasource and dashboard JSON files are mounted into Grafana's provisioning directory, eliminating manual UI setup.
- **OTA (Over-the-Air) Update:** A mechanism for distributing software updates wirelessly. In QuakeGuard, the React Native app polls GitHub Releases for new APK versions and triggers a native sideload prompt.

18. Appendix A: OpenAPI / REST Endpoints

The QuakeGuard Control Plane exposes a REST API via FastAPI. The full OpenAPI 3.0 specification is available at the `/docs` endpoint of the backend service. Below is a summary of the core endpoints:

Provisioning & Authentication

- `POST /devices/register`: Enrolls a new IoT node. Requires `enrollment_token`. Returns assigned `sensor_id` and zone metadata.

Ingestion (Data Plane Fallback)

- `POST /readings/`: Accepts ECDSA-signed seismic payloads. Used by the MQTT bridge and USB Serial fallback.

Alerts & Reports

- `GET /alerts/{zone_id}`: Retrieves the latest alerts for a specific geographic zone.
- `GET /reports/{alert_id}`: Fetches the AI-generated emergency report for a confirmed alert.

Telemetry & Analytics

- `GET /sensors/{sensor_id}/statistics`: Returns time-series aggregates (count, max magnitude) for a specific sensor, leveraging TimescaleDB continuous aggregates.

19. Appendix B: Project & Licensing Information

- **Software License:** QuakeGuard software (backend, mobile, firmware) is open-source and released under the GNU Affero General Public License v3.0 (AGPL-3.0). This ensures that any modifications or network use of the software remain freely available to the public.

- **Hardware License:** The QuakeGuard Printed Circuit Board (PCB) and related hardware designs are released under the CERN Open Hardware Licence (CERN-OHL-S).
- **Data Availability & Archival:** Stable releases of the project, including this whitepaper and the associated source code, are permanently archived on Zenodo and citable via the authoritative DOI identifier.

20. Founders & Core Maintainers

This project was conceived, designed, and built by two founders — responsible for the full-stack architecture from the ESP32-C3 edge firmware to the FastAPI/Redis/TimescaleDB backend and the React Native mobile client. Both maintain the codebase, hardware, and documentation as co-maintainers of the open-source organization.

20.1. Giovanni Zanotti — GiZano @GiZano

Founder, Systems & Edge Architecture

BSc Student in Computer Science @ University of Pisa (UniPi) — Edge Computing | Systems Architecture | Mathematical Foundations. Building robust IoT ecosystems, high-performance backends, and bridging the gap between low-level hardware constraints and cloud infrastructure. Currently deepening C/C++ & Linux internals and studying scalable IoT infrastructures.

- GitHub: github.com/GiZano
- Website: giovanni-zanotti.is-a.dev
- LinkedIn: linkedin.com/in/giovanni-zanotti-it
- ORCID: 0009-0000-8900-9586

20.2. Riccardo Dilecce — riccardo0731 @riccardo0731

Founder, Full-Stack & Mobile

Junior Software Developer & IT Student — passionate about Archery, Photography, and Full-Stack Development (Backend to Frontend). Focusing on Database (SQL, PHP) and Networking (Cisco), with hands-on experience across HTML/CSS/JavaScript, Java, Python, C++, Arduino, and Git.

- GitHub: github.com/riccardo0731
- Portfolio: riccardo0731.github.io
- LinkedIn: linkedin.com/in/riccardo-dilecce-archer